For example, analyzing the relationship between equipment age, maintenance frequency, and failure rates requires scanning years of maintenance records, device specifications, and sensor readings. Storing data in a columnar approach as specialized timeseries databases do also provides significant advantages for analytical queries where only specific columns need to be read rather than entire records.

ANALYTICAL DATABASES

Snowflake, Amazon Redshift, and Google BigQuery are prominent examples of cloud-native columnar data warehouses. They are purpose-built for analytical workloads, meaning they are optimized for complex queries, reporting, and business intelligence, rather than high-volume, low-latency transactional processing.

> ***NOTE***
>
> The separation of storage and compute in modern cloud analytical databases represents one of the most important innovations in cloud databases allowing independent scaling of each resource, reducing costs and improving flexibility. You can scale compute resources for complex queries while maintaining cost-effective storage for large datasets.

The main differences between these and dedicated timeseries databases (like InfluxDB or TimescaleDB) lies in their broader scope. While timeseries databases are focused on temporal data patterns and specific optimizations for time-based queries and compression, analytical data warehouses are designed to handle a wide variety of data types and structures, including structured relational data, semi-structured data, and of course timeseries data from other sources.

COLUMNAR FILE FORMATS

Specialized analytical databases like Redshift offer advantages such as complex SQL query capabilities (window functions, common table expressions), dedicated compute resources, and optimized query planning. However, if your workload doesn’t require these features, the additional cost may not be justified. If you have the need to perform occasional ad-hoc analytical queries from your digital twin, a file based columnar data store such as Apache Parquet is a cost efficient way to store data and still query it via SQL or Python. Another benefit of Parquet is that it is an open standard allowing you to move data between cloud providers or on-premises without vendor lock-in or data migration complexity. Listing 4.8 uses DuckDB to query taxi cab data in Parquet format using SQL, without the need to load the data to a database first.